It's too short...

Got it, let's make a **compact but not tiny** cheat sheet you can actually study from.

1. Core math of `nn.Linear`

Single input vector $x \in \mathbb{R}^{H_{in}}$:

$$y = xW^T + b$$

- $W \in \mathbb{R}^{H_{out} \times H_{in}}$
- $b \in \mathbb{R}^{H_{out}}$
- Output $y \in \mathbb{R}^{H_{out}}$

Per-coordinate:

$$y_j = \sum_{k=1}^{H_{in}} W_{j,k} x_k + b_j$$

Code:

```
layer = nn.Linear(H_in, H_out)
x = torch.randn(H_in)   # shape (H_in,)
y = layer(x)           # shape (H_out,)
```

2. Where `N` (batch) comes from

Instead of 1 input vector, you usually have **N inputs at once**:

- Stack them as rows in a matrix $X \in \mathbb{R}^{N \times H_{in}}$.
- Row n is $x^{(n)}$.

Layer applies same W, b to each row:

$$Y = XW^T + \mathbf{1}b^T$$

- $Y \in \mathbb{R}^{N \times H_{out}}$
- $\mathbf{1} \in \mathbb{R}^N$ is all ones (so bias is added to each row).

Code:

```
layer = nn.Linear(H_in, H_out)

N = 3
X = torch.randn(N, H_in)   # (3, H_in)
Y = layer(X)               # (3, H_out)
```

Interpretation: the first dimension is "how many examples at once", but the math per row is still $y = xW^T + b$.

3. What the `*` (extra structure) means

Now each example may have **structure**: sequence steps, positions, patches, etc.

Example: sequence of length T

- For each batch item n and time step t, we have $x^{(n,t)} \in \mathbb{R}^{H_{in}}$.
- Collect them into $X \in \mathbb{R}^{N \times T \times H_{in}}$.

Linear layer acts **independently** on every (n, t):

$$y^{(n,t)} = x^{(n,t)} W^T + b$$

Shapes:

- Input: `(N, T, H_in)`
- Output: `(N, T, H_out)`

Code:

```
layer = nn.Linear(H_in, H_out)

N, T = 2, 3
X = torch.randn(N, T, H_in)   # (2, 3, H_in)
Y = layer(X)                 # (2, 3, H_out)
```

So `(N, *, H_in)` is shorthand for:

- Some leading dimensions (N, maybe T, maybe others),
- Last dimension is the **feature vector**,
- Apply the same linear map to **every feature vector on the last axis**.

You can mentally think: *"Flatten all but the last dimension, apply the linear layer, reshape back."*

4. Training: math and code side-by-side

Say we do regression on a batch of size N.

Math:

- Predictions: $\hat{Y} = XW^T + b$
- Targets: T (same shape as $\hat{Y}$)
- Loss (mean squared error):

$$L = \frac{1}{N} \sum_{n=1}^{N} \|\hat{y}^{(n)} - t^{(n)}\|^2$$

Update rule, learning rate η:

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}, \quad b \leftarrow b - \eta \frac{\partial L}{\partial b}$$

Code:

```
H_in, H_out = 4, 1
layer = nn.Linear(H_in, H_out)
optimizer = torch.optim.SGD(layer.parameters(), lr=0.1)
loss_fn = nn.MSELoss()

for step in range(100):
    X = torch.randn(5, H_in)      # (N=5, H_in)
    target = torch.randn(5, H_out)

    pred = layer(X)               # (5, H_out)
    loss = loss_fn(pred, target)

    optimizer.zero_grad()
    loss.backward()               # computes dL/dW, dL/db
    optimizer.step()               # W, b get updated
```

You can imagine `X` being `(N, T, H_in)` as well; the same backward/update logic applies, just with extra dimensions carried along.

5. One concrete numeric example

Let:

- `H_in = 2`, `H_out = 1`
- One sample (no batch for now):

Weights:

- $W = [w_1, w_2]$
- $b = b$

Input $x = [x_1, x_2]$.

Then:

$$y = w_1 x_1 + w_2 x_2 + b$$

In PyTorch:

```
layer = nn.Linear(2, 1, bias=True)
with torch.no_grad():
    layer.weight[:] = torch.tensor([[1.0, 2.0]])   # w1=1, w2=2
    layer.bias[:]   = torch.tensor([0.5])          # b=0.5

x = torch.tensor([3.0, 4.0])   # x1=3, x2=4
y = layer(x)                  # y = 1*3 + 2*4 + 0.5 = 11.5
```

Add batch `N=2`:

- Inputs are `[[3,4], [1,2]]`:

$$\left[\begin{bmatrix} 3 & 4 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix} \begin{bmatrix} 11.5 \\ 5.5 \end{bmatrix} \right]$$

Code:

```
X = torch.tensor([[3.0, 4.0],
                  [1.0, 2.0]])   # (N=2, H_in=2)
Y = layer(X)                     # (2, 1) = [[11.5], [5.5]]
```

If that level of detail feels about right, next I can do one tiny `(N, T, H_in)` numeric example, or focus more on how to read these shapes quickly.